

Setup definitions in LoggerMessageDefines.txt

The API functions use the:

- ENABLE_LOGGING value,
- PRODUCTION value,
- logical message_group id=value and severity id=value pairs,

defined in \$LOGGER_HOME_PATH/LoggerSecureVault/LoggerMessageDefines.txt file as follows:

The ENABLE_LOGGING value must be 'y' to allow WandISSAL_LoggerAppsMsg() API to generate log messages and 'n' to not generate log message. If ENABLE_LOGGING is 'n' then API calls will simply return 0 without processing and generating any log message.

Do not change the keywords ENABLE_LOGGING_START and ENABLE_LOGGING_END.

```
*** Define the ENABLE_LOGGING to 'y' or 'n'.
*** If ENABLE_LOGGING=y then log messages will be generated.
*** If ENABLE_LOGGING is 'n' then log message will not be generated.
*** Do not change keywords ENABLE_LOGGING_START and ENABLE_LOGGER_END.
ENABLE_LOGGING_START
ENABLE_LOGGING=y
ENABLE_LOGGING_END
```

The PRODUCTION value is used to indicate what log messages are selected and generated in a production environment. If PRODUCTION=n then WandISSAL_LoggerAppsMsg() API will generate all log messages where DevProd argument is set to 1 or 2. If PRODUCTION=y then WandISSAL_LoggerAppsMsg() API will only log messages where DevProd=2, thus prevents generating unnecessary development log messages in a production environment.

Do not change keywords LOGGER_LOG_ENV_START and LOGGER_LOG_ENV_END.

```
*** If PRODUCTION=n then log messages with both DevProd=1 and
*** DevProd=2 will be generated.
*** If PRODUCTION=y then only log messages with DevProd=2
*** will be generated.
LOGGER_LOG_ENV_START
PRODUCTION=n
LOGGER_LOG_ENV_END
```

The logical message_group represents groups of related log messages that can be generated from one or more application functions within or across multiple application files. For example, defining a logical message_group id=value pair as 100=DATABASE and using 100 in the WandISSAL_LoggerAppsMsg() API will generate the value name DATABASE for those log messages. The logical message_group id integer part must be defined as a unique number (up to a maximum of 32 id=value pairs), and the value name part is an alphanumeric text that represents the message_group name (up to a maximum of 16 characters).

Do not change keywords LOGGER_MESSAGE_GROUP_START and LOGGER_MESSAGE_GROUP_END.

```
*** Define 1 to 32 unique logical message_group id=value pairs **/
LOGGER_MESSAGE_GROUP_START
100=DATABASE
101=User_Interface
```

```
102=Main_routine
LOGGER MESSAGE GROUP END
```

The `severity id=value` pair represents the importance for log messages. For example, severities for Fatal, Error, and Inform could be represented as 1, 2 and 3 respectively. The `severity id` integer part must be a unique number from 1 through 4 (a maximum of 4 `id=value` pairs), and the value name part is an alphanumeric text that represents a `severity` name (maximum of 16 characters).

Do not change keywords `LOGGER_SEVERITY_START` and `LOGGER_SEVERITY_END`.

```
/** Define from 1 through 4 severity id=value pairs */
LOGGER_SEVERITY_START
1=FATAL
2=ERROR
3=INFORM
LOGGER_SEVERITY_END
```

During `WandiSSAL_LoggerStartUp()` the above data values are validated and any error codes are written to the following files (where `<cpid>` is the application process id number):

- `/tmp/<cpid>-LoggerAppsMessage.txt`
- `$LOGGER_HOME_PATH/LoggerLibApps/LoggerOutput/<cpid>-LoggerAppsMessage.txt`

Note that lines that start with a “#” sign is treated as a commented line.